

Nettoyer un dataset sale

12 problèmes corrigés



Le symptôme qui trahit chaque problème, la méthode exacte en Power Query et en pandas, et le piège qui fausse ton rapport sans prévenir.

Face à un fichier brut, tu ne sais jamais par où commencer. Ce guide fixe l'ordre : 12 problèmes de données classés du plus visible au plus silencieux, chacun avec la méthode exacte dans les deux outils que tu utiliseras en poste.

Le premier fichier brut confié à un junior ressemble toujours au même scénario : le fichier « s'ouvre » sans erreur, le tableau croisé se calcule sans erreur, et pourtant le total ne colle pas. Ce n'est presque jamais un bug technique. C'est un problème de données que personne n'a signalé, parce qu'aucun de ces 12 problèmes ne fait planter Excel ni Python : ils faussent juste le résultat en silence.

Le réflexe qui distingue un junior confiant d'un junior qui espère : il inspecte son fichier avant de croire son fichier. C'est exactement ce que ce guide t'entraîne à faire.

Palier 1 : valeurs et types

On commence par le plus visible : une valeur qui manque ou un type qui ment. Si ces deux-là sont faux, tout calcul posé dessus l'est aussi.

Libellés Power Query en français, interface Excel 365 / Power BI Desktop : les noms de menu peuvent varier selon la version et le produit.

1. Valeurs manquantes (NaN, cellules vides) PIÈGE CLASSIQUE

Symptôme : des cellules vides ou des **NaN** dans une colonne censée être complète, invisibles tant que tu ne les comptes pas.

Power Query : Accueil > Supprimer les lignes > Supprimer les lignes vides, ou clic droit sur la colonne puis Remplacer les valeurs.

```
df["ca"].isna().sum()

# Option A : imputer par 0 (si l'absence de valeur veut dire "zero")
df["ca"] = df["ca"].fillna(0)

# Option B : exclure les lignes concernées (si la valeur est vraiment inconnue)
df = df.dropna(subset=["ca"])
```

Piège : la moyenne ignore les NaN par défaut, elle se calcule donc sur moins de lignes que tu ne crois, sans avertissement. C'est `fillna(0)` qui tire vraiment la moyenne vers le bas en ajoutant des zéros dans le calcul : choisis l'option selon ce que signifie l'absence de valeur, jamais les deux à la suite.

2. Colonnes numériques stockées en texte ÉLIMINATOIRE

Symptôme : la colonne se trie alphabétiquement (1, 10, 2, 20) au lieu de numériquement, et les nombres sont alignés à gauche.

Power Query : clic sur l'icône de type en tête de colonne (ABC/123) puis Nombre décimal ou Nombre entier.

```
df["prix_num"] = pd.to_numeric(df["prix"], errors="coerce")
```

Piège : `errors="coerce"` transforme les valeurs non convertibles en NaN sans le crier, un `sum()` ou un `mean()` derrière ignore ces lignes sans avertir.

3. Formats de date mixtes dans une même colonne ÉLIMINATOIRE

Symptôme : une colonne date qui mélange **05/01/2024** et **2024-01-05**, souvent parce que deux exports ont été concaténés.

Power Query : icône de type en tête de colonne > Date, puis fixer le format d'origine avant conversion (utilisation des paramètres régionaux).

```
est_iso = df["date"].str.contains("-", na=False)
df.loc[est_iso, "date_ok"] = pd.to_datetime(df.loc[est_iso, "date"], format="%Y-%m-%d")
df.loc[~est_iso, "date_ok"] = pd.to_datetime(df.loc[~est_iso, "date"], format="%d/%m/%Y")
```

Piège : un `pd.to_datetime(..., dayfirst=True)` sur toute la colonne ne passe pas en silence : depuis pandas 2.0, le format est deviné sur la première ligne puis imposé aux autres, et l'appel lève une `ValueError`. Et si tu ajoutes `format="mixed"` pour faire taire l'erreur, pandas relit alors 2024-01-05 comme le 1er mai. `errors="coerce"` tout court fait pire : les lignes qu'il ne reconnaît pas deviennent un NaT muet, sans erreur ni avertissement. Il faut séparer les deux formats avant de parser, pas les passer en bloc.

Ce que révèle le palier 1 : le silencieux avant le visible

Un fichier « sans erreur rouge » n'est pas un fichier propre. Chacun de ces trois cas se corrige en une ligne, mais seulement si tu l'as cherché.

Avant de croire un total

Compte les valeurs manquantes colonne par colonne :
`df.isna().sum()`.

Avant de croire un tri

Vérifie le type réel de chaque colonne, pas seulement son affichage.

Palier 2 : texte et formats

Le texte est le champion du silence : un espace, une majuscule ou un mauvais encodage ne lèvent aucune erreur, ils cassent juste tes jointures et tes groupes.

4. Espaces invisibles en début ou fin de chaîne PIÈGE CLASSIQUE

Symptôme : une jointure ou un filtre échoue sur des valeurs qui semblent identiques à l'œil, comme **"Paris"** et **"Paris"**.

Power Query : Transformer > Format > Ajuster (Trim), à appliquer sur la colonne concernée.

```
df["ville"] = df["ville"].str.strip()
```

Piège : un `VLOOKUP` ou un `merge` pandas ne matche jamais ces lignes et ne renvoie aucune erreur, juste des résultats vides ou des NaN qui passent inaperçus.

5. Casse incohérente (Paris / paris / PARIS)

Symptôme : un graphique par catégorie affiche trois barres pour une seule vraie catégorie.

Power Query : Transformer > Format > minuscules (ou MAJUSCULES / Mettre en majuscule chaque mot selon la cible).

```
df["ville_norm"] = df["ville"].str.lower()
df.groupby("ville_norm").size()
```

Piège : un `groupby` sur la colonne brute crée un faux doublon de catégorie par variante de casse, le total par ville reste juste, mais éclaté en plusieurs lignes.

Avant

"Paris" / "paris" / "PARIS" : trois clés distinctes pour un merge ou un groupby, alors qu'il n'y a qu'une seule vraie ville.

Après

`df["ville"].str.strip().str.lower()` : une clé unique et stable, prête à joindre ou regrouper.

6. Encodage de caractères mal décodé (accents, UTF-8) PIÈGE CLASSIQUE

Symptôme : des caractères parasites du type Ã© ou % apparaissent à la place des accents, typiquement après import d'un CSV exporté depuis un autre système.

Power Query : lors de l'import (Données > Depuis un fichier texte/CSV), régler l'origine du fichier sur le bon encodage (par exemple 65001 : Unicode (UTF-8)) avant de charger.

```
pd.read_csv("fichier.csv", encoding="utf-8")
pd.read_csv("fichier.csv", encoding="latin-1")
```

Piège : la variante mal décodée (Ã©) et la variante bien décodée créent deux catégories distinctes dans un groupby, l'une lisible et l'autre visiblement cassée : le total est éclaté en deux lignes au lieu d'une, sans qu'aucune erreur ne le signale.

7. Colonnes fusionnées à séparer (nom complet, adresse)

Symptôme : une colonne `nom_complet` contient prénom et nom ensemble, impossible à trier ou filtrer par nom seul.

Power Query : Transformer > Fractionner la colonne > Par délimiteur, en choisissant l'espace comme séparateur.

```
df[["prenom", "nom"]] = df["nom_complet"].str.split(" ", n=1, expand=True)
```

Piège : une ligne sans séparateur (un seul mot, comme un nom d'entreprise) se retrouve avec un NaN dans la seconde colonne, une perte d'information silencieuse si personne ne vérifie.

Ce que révèle le palier 2 : le texte ment plus que le chiffre

Un espace, une casse ou un octet mal décodé ne cassent jamais le calcul, ils cassent le regroupement derrière.

Avant un merge ou un VLOOKUP

Normalise les clés des deux côtés : `strip` puis `lower`.

Avant un groupby sur du texte

Compte les catégories distinctes, une variante de `drop_duplicates` est un signal.

Palier 3 : cohérence et doublons

Un fichier peut être propre en type et en texte, et compter deux fois la même ligne. Ici, le problème n'est plus la valeur, c'est le nombre de fois qu'elle apparaît.

8. Doublons exacts (lignes strictement identiques) ÉLIMINATOIRE

Symptôme : un total qui gonfle sans raison apparente après une fusion de plusieurs exports.

Power Query : clic droit sur la colonne ou la sélection de colonnes concernées, puis Supprimer les doublons.

```
df.duplicated().sum()
df = df.drop_duplicates()
```

Piège : des doublons créés par une fusion de sources en amont (le même client dans deux exports) passent inaperçus si tu ne les as pas comptés avant de sommer.

9. Doublons quasi identiques (variantes d'écriture) PIÈGE CLASSIQUE

Symptôme : "ACME Corp" et "acme corp" comptent comme deux clients différents dans un rapport.

Power Query : Accueil > Regrouper par sur une colonne normalisée créée au préalable (Ajouter une colonne > Colonne personnalisée en minuscule et ajustée).

```
df["client_norm"] = df["client"].str.strip().str.lower()
df["client_norm"].duplicated().sum()
df_clients = df.drop_duplicates(subset="client_norm")
```

Piège : un `drop_duplicates` brut sur la colonne d'origine ne détecte rien, puisque les deux chaînes sont techniquement différentes. Et cette normalisation ne vaut que sur un référentiel client, une ligne par client : sur une table de commandes ou de transactions, elle supprimerait à tort les achats suivants du même client.

10. Valeurs aberrantes (outliers) ÉLIMINATOIRE

Symptôme : une moyenne étrangement haute, alors que la majorité des lignes semblent normales.

Power Query : flèche de filtre de la colonne numérique > Filtres numériques > Entre, avec des bornes calculées à la main.

```
q1, q3 = df["montant"].quantile([0.25, 0.75])
iqr = q3 - q1
bas, haut = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df[(df["montant"] < bas) | (df["montant"] > haut)]
```

Piège : la moyenne masque l'aberration au lieu de la révéler, une seule ligne à 10 000 dans une série autour de 100 fait paraître le tout légèrement élevé, sans jamais crier l'anomalie. Le seuil 1,5 x IQR est une règle conventionnelle, pas une vérité universelle : à arbitrer avec un seuil métier selon ton contexte.

Ce que révèle le palier 3 : un chiffre juste peut venir d'un total faux

Le calcul est correct, ce sont les lignes en entrée qui ne le sont pas. La correction se fait toujours avant l'agrégation, jamais après.

Avant de sommer

Compare le nombre de lignes attendu au nombre de lignes réel.

Avant de moyenner

Regarde la distribution, pas seulement la moyenne.

Palier 4 : unités et catégories

Dernier palier, le plus fin : une unité mélangée ou une catégorie mal orthographiée ne casse rien techniquement, mais fausse la lecture métier.

11. Unités incohérentes dans une même colonne (kg/g, €/€) PIÈGE CLASSIQUE

Symptôme : un écart d'ordre de grandeur suspect entre lignes, un poids de **1.2** à côté d'un poids de **500** dans la même colonne.

Power Query : Ajouter une colonne > Colonne conditionnelle, règle : si Unité = g alors valeur divisée par 1000, sinon valeur inchangée.

```
df["poids_kg"] = np.where(df["unite"] == "g", df["poids"] / 1000, df["poids"])
```

Piège : une somme sur la colonne brute mélange kilogrammes et grammes sans aucun avertissement, le total n'a alors aucun sens physique.

12. Catégories mal orthographiées ou incohérentes

Symptôme : un graphique par catégorie affiche beaucoup trop de barres pour un nombre de catégories métier réellement attendu.

Power Query : clic droit sur la colonne > Remplacer les valeurs (une paire à la fois), ou fusion avec une table de correspondance via Accueil > Fusionner les requêtes.

```
mapping = {"france": "France", "fr": "France", "francia": "France"}  
df["pays_norm"] = df["pays"].str.lower().replace(mapping)
```

Piège : une segmentation ou un graphique éclate une même catégorie métier en plusieurs barres, ce qui fait paraître chaque part plus petite qu'elle ne l'est réellement.

Ce que révèle le palier 4 : l'unité et la catégorie sont des hypothèses, pas des faits

Rien ne plante techniquement sur ces deux cas. C'est la lecture métier qui se déforme, souvent découverte trop tard, en réunion.

Avant de sommer une colonne physique

Vérifie qu'une seule unité est utilisée sur toute la colonne.

Avant un graphique par catégorie

Compte les valeurs distinctes, une liste trop longue cache des doublons de sens.

Dans quel ordre traiter les 12 cas sur un vrai fichier

Cet ordre n'est pas une règle officielle, c'est la pratique de terrain qui évite de refaire un nettoyage deux fois.

1. Types et valeurs manquantes d'abord (problèmes 1 à 3) : tant qu'une colonne numérique est en texte ou qu'une date est ambiguë, aucun calcul suivant n'est fiable.
2. Texte et formats ensuite (problèmes 4 à 7) : espaces, casse, encodage, colonnes fusionnées. Ils conditionnent tes futures jointures et regroupements.
3. Doublons et cohérence (problèmes 8 à 10) : une fois les valeurs et le texte fiables, tu peux enfin faire confiance au nombre de lignes.
4. Unités et catégories en dernier (problèmes 11 et 12) : ce sont des corrections de lecture métier, elles s'appliquent sur des données déjà propres.

Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat sur son premier fichier brut. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

